



Hızlı Başvuru Kağıdı #2: Kontrol Yapıları, Fonksiyonlar ve Veri Yapıları

PyGame ile Oyun Geliştirme – Hafta 3–4

Koşullu İfadeler

if / elif / else

```
if kosul1:
    # kosul1 dogru ise
elif kosul2:
    # kosul2 dogru ise
else:
    # hicbiri degilse
```

Örnek

```
puan = 85
if puan >= 90:
    not_ = "A"
elif puan >= 80:
    not_ = "B"
elif puan >= 70:
    not_ = "C"
else:
    not_ = "F"
```

Dikkat

= atama, == karşılaştırma! if x = 5 hata verir.

Döngüler

while Döngüsü

```
while kosul:
    # kosul dogru oldugu süreç
    # (sonsuz donguye dikkat!)
```

for Döngüsü

```
for eleman in koleksiyon:
    # her eleman için çalışır

for i, deger in enumerate(liste):
    # indeks + deger birlikte
```

Fonksiyonu

Kullanım	Sonuç
range(5)	0,1,2,3,4
range(2, 6)	2,3,4,5
range(0, 10, 2)	0,2,4,6,8
range(5, 0, -1)	5,4,3,2,1

ve

break Döngüyü tamamen sonlandır
continue Bu adımı atla, sonrakine geç

Kalıp: Sonsuz Döngü + break

```
while True:
    veri = input("Gir (q=cik): ")
    if veri == "q":
        break
```

Fonksiyonlar

İle Tanımlama

```
def fonksiyon_adi(param1, param2):
    """Docstring: ne yapar."""
    # islemler
    return sonuc
```

Parametre Türleri

Pozisyonel

```
def topla(a, b):
    return a + b
```

Varsayılan değerli

```
def selamla(ad, mesaj="Merhaba"):
    print(f"{mesaj}, {ad}!")
```

Keyword arguman

```
selamla(mesaj="Selam", ad="Ali")
```

İfadesi

Tek değer return sonuc
Çoklu değer return x, y (tuple döner)
Değer yok return veya hiç yazma → None

Çoklu Dönüş Değeri

```
def bolme(a, b):
    bölüm = a // b
    kalan = a % b
    return bölüm, kalan
```

```
b, k = bolme(17, 5) # b=3, k=2
```

Kapsam (Scope)

Local vs Global

- Fonksiyon içi değişkenler **local** (dışarıdan erişilemez)
- Fonksiyon dışı değişkenler **global** (okuma serbest)
- Global değişkeni değiştirmek için **global** x gerekir

Lambda Fonksiyonları

Tek Satır Fonksiyon

```
kare = lambda x: x ** 2
print(kare(5)) # 25
```

```
topla = lambda a, b: a + b
print(topla(3, 4)) # 7
```

Listeler

Oluşturma ve Erişim

```
liste = [1, 2, 3, 4, 5]
liste[0] # 1 (ilk eleman)
liste[-1] # 5 (son eleman)
liste[1:3] # [2, 3] (dilimleme)
liste[:2] # [1, 2]
liste[::2] # [1, 3, 5] (adım 2)
```

Metot

Metot	Açıklama
.append(x)	Sona ekle
.insert(i, x)	i konumuna ekle
.remove(x)	İlk x'i sil
.pop(i)	i konumundan çıkar (varsayılan: son)
.sort()	Yerinde sırala
.reverse()	Ters çevir
.index(x)	x'in konumu
len(liste)	Eleman sayısı
x in liste	Üyelik testi

Sözlükler

Oluşturma ve Erişim

```
oyuncu = {
    "isim": "Kahraman",
    "can": 100,
    "envanter": ["kilic"]
}
oyuncu["can"] # 100
oyuncu.get("mana", 0) # 0 (yoksa)
oyuncu["guc"] = 25 # yeni ekle
```

Metot

Metot	Açıklama
.keys()	Tüm anahtarlar
.values()	Tüm değerler
.items()	(anahtar, değer) çiftleri
.pop(k)	Anahtarı sil ve döndür
.update(d)	Başka sözlükle birleştir
.get(k, v)	Güvenli erişim (yoksa v)

Demetler ve Kalıplar

Tuple: Değiştirilemez

```
konum = (10, 20)
x, y = konum # unpacking
```

Yaygın Döngü Kalıpları

```
# Sayac
sayac = sum(1 for x in liste if x > 10)
# Toplam
toplam = sum(liste)
# Arama
for k, v in sozluk.items():
    if v == aranan: print(k)
```